1. Situación de partida

Una pequeña empresa dedicada al desarrollo web quiere crear una página informativa denominada DAW Project Hub. La primera versión será sencilla: presentará tres textos breves sobre el desarrollo web, el despliegue de aplicaciones y el trabajo colaborativo.

La página debe mantenerse mediante Git y publicarse en GitHub. Cada modificación tendrá que quedar registrada, las mejoras de contenido o de presentación se desarrollarán en ramas independientes y las propuestas externas se incorporarán mediante forks, incidencias y pull requests.

El objetivo no consiste solamente en terminar la página web. El resultado debe mostrar un proceso de desarrollo ordenado, trazable, seguro y colaborativo.

2. Objetivos de la práctica

- Configurar Git y conectarlo con GitHub mediante una clave SSH.
- Distinguir el directorio de trabajo, el área de preparación y el repositorio local.
- Utilizar status, diff, add, commit y log para registrar y consultar cambios.
- Excluir archivos mediante .gitignore y evitar la publicación de información sensible.
- Desarrollar cambios de contenido y presentación en ramas independientes y fusionarlas.
- Provocar, interpretar y resolver un conflicto de fusión.
- Trabajar con repositorios remotos mediante clone, fetch, pull y push.
- Investigar y utilizar issues, forks, remotos upstream y pull requests.
- Revisar propuestas, solicitar cambios y actualizar una pull request.
- Identificar una versión estable mediante una etiqueta y una release.

3. Organización

La primera parte se realizará individualmente. La fase de colaboración se realizará por parejas. Cada participante tendrá un repositorio propio, una copia local del proyecto, una cuenta de GitHub, una clave SSH configurada y una persona con quien intercambiar propuestas.

4. Producto que se debe desarrollar

Se creará una pequeña página web denominada DAW Project Hub con esta estructura:

```
daw-project-hub/
├── index.html
├── css/
│   └── styles.css
├── .gitignore
├── .env.example
└── README.md
```

La página debe contener, como mínimo:

- Una cabecera con el nombre DAW Project Hub y una breve presentación.
- Una sección principal formada por tres párrafos de contenido.
- Un primer párrafo sobre el desarrollo web, un segundo párrafo sobre el despliegue y un tercer párrafo sobre el trabajo colaborativo.
- Un título sencillo para la sección principal.
- Un pie de página y un diseño adaptable básico.

La página se realizará únicamente con HTML y CSS. No se utilizará JavaScript ni se exigirán componentes avanzados como tarjetas, filtros o elementos interactivos.

No se valorará especialmente la complejidad visual. Se valorará que los cambios se incorporen mediante un proceso correcto de control de versiones.

Fase 1. Preparación de Git y conexión con GitHub

Tarea 1. Comprobar la instalación

Comprobar que Git está instalado y consultar su versión. Configurar el nombre de autor, la dirección de correo asociada a GitHub y main como nombre predeterminado de la rama principal.

Registrar en el README.md la versión de Git, el sistema operativo y el editor de código utilizados.

Tarea 2. Investigar la autenticación mediante SSH

Antes de configurar la conexión, investigar y responder brevemente en el README.md:

1. ¿Qué es SSH?
2. ¿Qué diferencia existe entre una clave pública y una clave privada?
3. ¿Qué clave puede compartirse?
4. ¿Qué clave no debe compartirse nunca?
5. ¿Qué ventaja presenta SSH frente a introducir continuamente usuario y contraseña?
6. ¿Qué función cumple el archivo known_hosts?
7. ¿Qué podría ocurrir si una clave privada se publica en GitHub?

Las respuestas deben redactarse con palabras propias.

Tarea 3. Crear y configurar una clave SSH

1. Comprobar si ya existe una pareja de claves SSH válida.
2. Si no existe, generar una nueva clave.
3. Proteger la clave privada con una frase de paso.
4. Añadir la clave pública a la cuenta de GitHub.
5. Comprobar que la autenticación funciona y que GitHub reconoce la cuenta.

La clave privada no debe subirse al repositorio, copiarse al README.md, incluirse en capturas, enviarse a otra persona ni almacenarse dentro del proyecto.

En el README.md solo se indicará:

```
Conexión SSH con GitHub: comprobada correctamente
```

Fase 2. Creación del repositorio local

Tarea 4. Inicializar el proyecto

Crear la carpeta daw-project-hub e inicializar en ella un repositorio Git. Antes de crear ningún archivo, comprobar la rama activa, el estado del repositorio, si existe algún commit y la ubicación del directorio de trabajo.

Crear index.html y README.md. El contenido inicial del README.md será:

```
# DAW Project Hub

Pequeña página web para practicar un flujo profesional de trabajo con Git y GitHub.
```

Realizar el primer commit con un mensaje claro. Este primer commit solo debe incluir los dos archivos anteriores.

Tarea 5. Construir la versión inicial mediante commits pequeños

Crear la página progresivamente. No debe incorporarse todo en un único commit. Se realizarán, al menos, estos cambios independientes:

1. Crear la estructura HTML general.
2. Añadir la cabecera y la presentación.
3. Añadir la sección principal.
4. Escribir los tres párrafos de contenido.
5. Incorporar el pie de página.
6. Crear la hoja de estilos.
7. Aplicar los estilos básicos.

Antes de cada commit se deberá consultar el estado, revisar las diferencias, preparar únicamente los archivos relacionados, volver a consultar el estado y crear el commit.

Los mensajes deben describir qué se ha hecho. No se admitirán mensajes genéricos como cambios, cosas, prueba, actualización o commit nuevo.

```
Añade la estructura principal de la página
Incorpora los tres párrafos de contenido
Aplica los estilos iniciales
Añade el pie de página
```

Tarea 6. Examinar el historial

Cuando exista un mínimo de cinco commits, consultar el historial completo, el historial resumido y el gráfico de ramas. Identificar el commit que añadió los tres párrafos y examinar sus cambios.

Añadir al README.md una sección denominada Historial del proyecto y explicar por qué resulta preferible realizar varios commits pequeños y coherentes en lugar de un único commit con toda la página.

Fase 3. Archivos ignorados y protección de información

Tarea 7. Configurar .gitignore

Crear localmente los archivos debug.log, .env y notas-temporales.txt. El archivo .env contendrá datos ficticios:

```
API_URL=http://localhost:3000
API_TOKEN=token-de-prueba
```

Configurar .gitignore para excluir los archivos terminados en .log, el archivo .env, notas-temporales.txt y las carpetas .vscode e .idea. Comprobar que no se incorporan al repositorio.

Crear también .env.example, que sí debe incorporarse al repositorio:

```
API_URL=
API_TOKEN=
```

Realizar un commit dedicado exclusivamente a esta configuración.

Reflexión de seguridad

1. ¿Por qué .env.example puede publicarse?
2. ¿Por qué .env debe ignorarse?
3. ¿Qué habría que hacer si una contraseña o un token reales se hubieran publicado en GitHub?
4. ¿Bastaría con eliminar el archivo en un commit posterior? Justificar la respuesta.

Fase 4. Desarrollo mediante ramas

Tarea 8. Crear una rama para mejorar la presentación

Desde una rama main actualizada, crear la rama:

```
feature/estilos-texto
```

1. Añadir clases CSS a los tres párrafos para poder identificarlos.
2. Mejorar la tipografía, el interlineado y los márgenes del contenido.
3. Aplicar un color de borde diferente a cada párrafo.
4. Realizar al menos dos commits.
5. Comprobar que main todavía no contiene las mejoras.
6. Revisar las diferencias entre ambas ramas.
7. Fusionar la rama en main.
8. Comprobar el funcionamiento y eliminar la rama local una vez integrada.

No se debe trabajar directamente en main mientras se desarrolla esta mejora.

Tarea 9. Crear una rama para ampliar el contenido

Crear feature/contenido-adicional. Añadir un subtítulo antes de cada uno de los tres párrafos: Desarrollo web, Despliegue y Colaboración.

Realizar al menos dos commits: uno para modificar el contenido HTML y otro para ajustar los estilos de los subtítulos. Fusionar la rama en main después de comprobar el resultado.

Comprobación

El historial deberá mostrar claramente la rama principal, los commits de las dos ramas de mejora y su incorporación a main.

Fase 5. Creación y resolución de un conflicto

Tarea 10. Provocar un conflicto controlado

El objetivo es generar un conflicto real, analizarlo y resolverlo sin eliminar el repositorio ni descartar todo el trabajo.

1. Desde main, crear feature/nuevo-eslogan.
2. En esa rama, cambiar la presentación por: Un espacio para aprender a desarrollar, publicar y mantener aplicaciones web.
3. Confirmar el cambio mediante un commit.
4. Volver a main.
5. Modificar exactamente la misma línea por: Una página para conocer las fases del desarrollo y despliegue de una aplicación web.
6. Confirmar el cambio en main.
7. Intentar fusionar feature/nuevo-eslogan en main.

Al aparecer el conflicto, consultar el estado, identificar el archivo afectado, localizar e interpretar los marcadores, redactar una versión final que combine las dos ideas, eliminar los marcadores, marcar el conflicto como resuelto, completar la fusión y revisar el historial.

Añadir al README.md una sección Conflicto resuelto que explique por qué se produjo, qué archivo estaba afectado, qué decisión se tomó y cómo se comprobó la resolución.

Fase 6. Publicación en GitHub

Tarea 11. Crear el repositorio remoto

Crear en GitHub un repositorio denominado daw-project-hub. No inicializarlo con README.md, .gitignore ni licencia. Utilizar la URL SSH del repositorio.

Conectar el repositorio local y publicar main. Comprobar que los archivos esperados aparecen, que .env y debug.log no aparecen, que el historial se conserva, que la conexión utiliza SSH y que la rama local sigue a la rama remota.

Tarea 12. Trabajar con ramas remotas

Crear localmente docs/instrucciones. Ampliar el README.md con instrucciones para abrir la página, publicar la rama y comprobar que aparece en GitHub.

1. Comparar la rama con main.
2. Integrar los cambios.
3. Publicar la nueva versión de main.
4. Eliminar la rama local.
5. Eliminar la rama remota.

Fase 7. Sincronización entre repositorios

Tarea 13. Clonar el proyecto

Clonar el repositorio en otra ubicación con el nombre daw-project-hub-copia. Comprobar que contiene los archivos, el historial, la configuración del remoto y la rama principal. No se debe copiar manualmente la carpeta original.

Tarea 14. Diferenciar fetch y pull

Desde la interfaz web de GitHub, modificar el README.md y crear un commit remoto con el mensaje Actualiza la descripción desde GitHub.

1. Comprobar en la copia local que el cambio todavía no aparece.
2. Descargar la información del remoto sin integrarla.
3. Consultar las diferencias entre la rama local y la remota.
4. Integrar finalmente el cambio.
5. Comprobar que el archivo local se ha actualizado.

Responder en el README.md qué hizo fetch, qué hizo pull, por qué fetch permite revisar antes de integrar y qué relación existe entre pull, fetch y la integración posterior.

Fase 8. Investigación sobre forks y propuestas de mejora

Tarea 15. Investigar el concepto de fork

Añadir al README.md la sección Forks y colaboración y responder con palabras propias:

1. ¿Qué es un fork en GitHub?
2. ¿En qué se diferencia un fork de una rama?
3. ¿En qué cuenta se almacena un fork?
4. ¿Cuándo resulta útil trabajar mediante un fork?
5. ¿Qué relación existe entre el repositorio original y el fork?
6. ¿Qué es el repositorio upstream?
7. ¿Qué diferencia existe entre origin y upstream?
8. ¿Cómo se propone que un cambio del fork llegue al repositorio original?
9. ¿Quién decide si se acepta la propuesta?
10. ¿Puede seguir evolucionando el repositorio original mientras existe el fork?

No se admitirán definiciones copiadas literalmente sin explicación.

Fase 9. Colaboración mediante issues, forks y pull requests

Esta fase se realizará por parejas. Cada participante será responsable de su propio repositorio y colaborador externo en el repositorio de otra persona.

Tarea 16. Proponer una mejora mediante una issue

Revisar el repositorio de la pareja y crear una issue con título, descripción, justificación, resultado esperado y criterios de comprobación. La mejora puede consistir en corregir o ampliar un párrafo, añadir un enlace, incorporar una lista sencilla, mejorar un título, ajustar márgenes, mejorar el contraste, cambiar un color o corregir un error ortográfico.

No debe comenzarse la modificación hasta que la persona responsable haya revisado la propuesta.

Tarea 17. Crear un fork

1. Crear un fork del repositorio de la pareja.
2. Clonar el fork mediante SSH.
3. Comprobar cuál es origin.
4. Añadir el repositorio original como upstream.
5. Consultar los remotos configurados.
6. Actualizar el fork con la versión más reciente del original.
7. Crear una rama específica para la propuesta.

```
feature/nombre-mejora
fix/nombre-correccion
docs/nombre-documentacion
```

No se debe realizar el cambio directamente sobre main.

Tarea 18. Implementar la propuesta

1. Realizar la modificación acordada en la rama.
2. Revisar las diferencias.
3. Comprobar el funcionamiento.
4. Realizar uno o varios commits coherentes.
5. Publicar la rama en el fork.

Tarea 19. Crear una pull request

Abrir una pull request desde el fork hacia main del repositorio original. Incluir título, resumen, motivo, archivos afectados, comprobaciones realizadas y referencia a la issue. Debe quedar claro el origen, el destino y el problema resuelto.

Tarea 20. Revisar la propuesta

1. Leer la pull request y examinar los archivos modificados.
2. Revisar las diferencias y comprobar que no contiene archivos privados.
3. Publicar al menos un comentario de revisión.
4. Solicitar un pequeño cambio antes de aprobarla.

Tarea 21. Actualizar la pull request

1. Aplicar el cambio solicitado en la misma rama.
2. Crear un nuevo commit.
3. Publicar el commit en el fork.
4. Comprobar que la pull request se actualiza automáticamente.
5. Responder al comentario de revisión.

No debe cerrarse la pull request ni crearse otra nueva.

Tarea 22. Aceptar e integrar la propuesta

1. Aprobar la pull request.
2. Integrarla en main.
3. Cerrar la issue vinculada.
4. Actualizar el repositorio local.
5. Comprobar que el cambio externo aparece en la página.
6. Eliminar las ramas que ya no sean necesarias.

Fase 10. Creación de una versión

Tarea 23. Preparar la versión 1.0.0

Antes de marcar la versión, comprobar que main está actualizada, no existen cambios sin confirmar, la página funciona, no hay archivos privados publicados y la pull request está integrada.

Crear una etiqueta anotada:

```
v1.0.0
Primera versión estable de DAW Project Hub
```

Publicar la etiqueta y crear una release que explique el contenido incluido, las mejoras recibidas, el estado de la página y las limitaciones conocidas.

Resultado final esperado

- Página web informativa terminada y un mínimo de diez commits significativos.
- Archivo .gitignore y archivo .env.example, sin secretos ni credenciales.
- Rama principal main, historial de ramas y fusiones y un conflicto resuelto.
- Una issue creada por otra persona y otra propuesta en un repositorio externo.
- Un fork realizado y un remoto upstream configurado.
- Una pull request enviada y otra recibida.
- Una revisión con solicitud de cambios y una actualización de la propuesta.
- Etiqueta v1.0.0, release y README.md completo.

Contenido mínimo del README.md

```
# DAW Project Hub

## Descripción
## Tecnologías utilizadas
## Cómo abrir la página
## Historial del proyecto
## Seguridad y archivos ignorados
## Conflicto resuelto
## Diferencias entre fetch y pull
## Forks y colaboración
## Colaboración realizada
## Autoría
```

Entrega

1. URL del repositorio propio.
2. URL de la página publicada, si se ha utilizado GitHub Pages.
3. URL de la issue creada en el repositorio de la pareja.
4. URL del fork realizado.
5. URL de la pull request enviada.
6. URL de la pull request recibida.
7. Nombre de la rama utilizada para la colaboración.
8. Nombre de la etiqueta final.
9. Explicación breve del conflicto resuelto.
10. Confirmación de que no se ha publicado ninguna clave, contraseña o token.

No es necesario incluir capturas cuando la acción pueda comprobarse directamente en GitHub.

Normas importantes

- No se puede borrar el repositorio y comenzar de nuevo para evitar un conflicto.
- No se puede trabajar exclusivamente sobre main.
- No se puede subir una clave privada, contraseña, token o archivo .env.
- No se puede copiar manualmente una carpeta para sustituir git clone.
- No se puede descargar un ZIP para sustituir un fork o una clonación.
- No se puede crear una segunda pull request para evitar corregir la primera.
- Los commits deben describir cambios concretos.

- La propuesta externa debe desarrollarse en una rama.
- La versión final debe poder reconstruirse a partir del repositorio.

Criterios de evaluación

Aspecto	Puntuación
Configuración de Git y conexión SSH segura	1,00
Estados, diferencias, preparación y commits	1,50
Calidad y organización del historial	1,00
Uso de .gitignore y protección de información	1,00
Creación, utilización y fusión de ramas	1,50
Resolución razonada del conflicto	1,00
Remotos y uso de fetch, pull y push	1,00
Investigación y utilización de fork y upstream	0,75
Issue, pull request, revisión y actualización	1,00
Etiqueta, release y calidad de la entrega	0,25
TOTAL	**10,00**

Penalizaciones directas

- Entregar todo el trabajo en uno o dos commits: no se considerará demostrado el uso progresivo de Git.
- No utilizar ramas: no se considerará superada la parte correspondiente.
- No colaborar mediante fork y pull request: no se considerará demostrada la colaboración externa.
- Copiar definiciones sin explicarlas: la investigación se considerará incompleta.